

ActSim Manual

A Python-Based Actuarial Risk Modelling and Simulation Package

Version 0.1.0

Juntao Zhang

May 25, 2026

*To my daughter, Yangling,
for being a blessing in my life.*

Abstract

ActSim is a Python-based actuarial simulation package designed for distribution fitting, stochastic risk modelling, synthetic claim generation, and actuarial validation and stress testing workflows. It is intended to support actuarial risk analysis, catastrophe scenario testing, capital adequacy testing, loss scenario analysis, and integration with triangle-based reserving workflows.

This manual provides a structured reference for users, including a user guide, worked examples, API reference, validation notes, and contribution guidance. ActSim emphasises actuarial transparency, reproducibility, and practical applicability for both research and production environments.

Contents

1	Introduction	5
1.1	What is ActSim?	5
1.2	Why ActSim Exists	5
1.3	Capabilities at a Glance	6
1.4	Positioning Statement	6
1.5	Audience	6
1.6	How to Read This Manual	6
2	Installation	7
2.1	Requirements	7
2.2	Standard Installation	7
2.3	Installation from Source	7
2.4	Development Installation	8
2.5	Recommended Virtual Environment	8
2.6	Verifying the Installation	8
3	Quick Start	9
3.1	Minimal Example	9
3.2	What the Example Does	9
3.3	Next Steps	10
4	Core Concepts	11
4.1	Frequency Distributions	11
4.2	Severity Distributions	11
4.3	Aggregate Loss	12
4.4	Dependency and Correlation	12
4.5	Claim-Level Simulation	12
4.6	Claim Development	13
4.7	Risk Metrics	13
5	ActStats	14
5.1	Overview	14
5.2	Why a Dedicated Actuarial Wrapper?	14
5.3	Supported Distributions	14
5.4	Parameter Conventions	15
5.5	Common Operations	15
5.6	Discrete Distributions	15

5.7	Non-Homogeneous Poisson Process	15
5.8	Date Conversion Helpers	16
6	DistributionFitter	17
6.1	Purpose	17
6.2	Supported Distributions	17
6.3	Goodness-of-Fit Metrics	17
6.4	Basic Workflow	18
6.5	Inspecting Results	18
6.6	Manually Selecting a Distribution	19
6.7	Sampling and Prediction	19
6.8	Diagnostic Plots	19
6.9	Summary Reports	19
6.10	Practical Notes	20
7	StochasticSimulator	21
7.1	Purpose	21
7.2	Construction	21
7.3	Frequency–Severity Simulation	22
7.4	Dependency between Frequency and Severity	22
7.4.1	Linear Correlation	22
7.4.2	Copula-Based Dependency	23
7.5	Multivariate Correlated Simulation	23
7.6	Analysing Results	23
7.6.1	Aggregate Percentiles	23
7.6.2	Risk Metrics	24
7.6.3	Distribution Plot	24
7.6.4	Correlation Plot	24
7.7	Applying Deductibles and Limits	24
7.8	Reproducibility	25
8	ClaimSimulator	26
8.1	Purpose	26
8.2	Required Policy Schema	26
8.3	Construction	26
8.4	Simulating Claims	27
8.4.1	Policies with Zero Simulated Claims	27
8.5	Simulating Claim Dates	27
8.6	Simulating Claim Development	28
8.6.1	Output Schema	29
8.7	Saving Claim Development Data	29
9	Risk Metrics	30
9.1	Overview	30
9.2	Mean and Standard Deviation	30
9.3	Percentiles	30
9.4	Value at Risk (VaR)	31
9.5	Tail Value at Risk (TVaR)	31

9.6	Aggregate Exceedance Probability (AEP)	31
9.7	Occurrence Exceedance Probability (OEP)	31
9.8	Putting it Together	32
9.9	Choice of Metric	32
10	Integration with Chainladder	33
10.1	Overview	33
10.2	From Simulated Claims to a Triangle	33
10.3	Constructing a chainladder Triangle	34
10.4	Position of ActSim Relative to Chainladder	34
11	Worked Examples	36
11.1	Example 1: Aggregate Loss Simulation	36
11.2	Example 2: Fitting Severity Distributions to Historical Losses	37
11.3	Example 3: Synthetic Policy and Claim Generation	38
11.4	Example 4: Developing Claims Using LDFs	38
11.5	Example 5: Converting Simulated Claims to a Triangle	39
11.6	Example 6: Analysing the Triangle with chainladder	39
11.7	Example 7: Calculating VaR and TVaR	40
11.8	Example 8: Validating Simulated Frequency	40
12	Validation and Testing	42
12.1	Why Validation Matters	42
12.2	Frequency Distribution Validation	42
12.3	Severity Distribution Validation	42
12.4	Aggregate Loss Validation	43
12.5	Claim Development Validation	43
12.6	Triangle Reconciliation	44
12.7	Reproducibility Using Random Seeds	44
12.8	Statistical Tests	45
12.9	Known Limitations	45
13	API Reference	46
13.1	DistributionFitter	46
13.2	StochasticSimulator	47
13.3	ClaimSimulator	48
13.4	Config and load_config	49
13.5	Utility Functions	49
13.6	Supported Distribution Names	50
14	Contributing	51
14.1	Scope of Contributions	51
14.2	Setting Up a Development Environment	51
14.3	Running the Test Suite	52
14.4	Code Style	52
14.5	Adding a New Distribution	52
14.6	Pull Request Workflow	52
14.7	Reporting Issues	53

14.8 Discussion and Roadmap	53
15 Citation	54
15.1 Citing ActSim	54
15.2 Citing Dependencies	54
16 Glossary	56
17 Appendix	58
17.1 Supported Distributions	58
17.2 Parameter Conventions	59
17.3 Example Input Schemas	59
17.4 Default Configuration File	60
17.5 Simulation Assumptions	61
17.6 Known Limitations	61
17.7 Release History	61

Chapter 1

Introduction

1.1 What is ActSim?

ActSim is a Python-based actuarial simulation package designed for loss simulation, stochastic risk modelling, synthetic claim generation, and actuarial validation workflows. The package provides a transparent and extensible framework for actuaries, researchers, and analysts who need to generate, analyse, and validate simulated insurance loss data.

The package is intended to modernise and extend traditional actuarial loss simulation tools by providing a Python-native framework that integrates with modern actuarial analytics workflows, including `pandas`, `NumPy`, `SciPy`, `chainladder`, and LLM-assisted actuarial tools.

1.2 Why ActSim Exists

Actuarial workflows have historically depended on a mix of legacy spreadsheet models, vendor software, and bespoke scripts. While these approaches remain valuable, they often suffer from limited reproducibility, opaque assumptions, and weak integration with the broader scientific Python ecosystem.

ActSim addresses several practical needs:

- Provide a Python-native framework for frequency-severity simulation that can be inspected, audited, and extended.
- Offer a reproducible workflow for synthetic claim-level data generation, suitable for testing reserving methods and stress scenarios.
- Support claim development through user-supplied loss development factors, with stochastic variation around expected patterns.
- Integrate cleanly with triangle-based reserving tools such as the `chainladder` package, so that simulated data can be analysed using established reserving methods.
- Provide standard risk metrics such as Value at Risk (VaR), Tail Value at Risk (TVaR), Aggregate Exceedance Probability (AEP), and Occurrence Exceedance Probability (OEP).

1.3 Capabilities at a Glance

ActSim supports the following capabilities:

- Frequency–severity simulation using a wide range of standard distributions.
- Aggregate loss modelling with optional dependency between frequency and severity, including linear correlation and copula structures.
- Multivariate correlated simulation across multiple lines of business.
- Claim-level synthetic data generation from a policy table, including occurrence date assignment via a non-homogeneous Poisson process.
- Claim development using user-defined loss development factors with stochastic volatility.
- Risk metric calculation, including VaR, TVaR, AEP, and OEP.
- Validation and testing workflows, with reproducibility ensured via explicit random seeds.
- Integration with triangle-based reserving tools.

1.4 Positioning Statement

ActSim is intended to support actuarial simulation workflows, including frequency–severity modelling, stochastic aggregate loss simulation, claim-level synthetic data generation, claim development, and integration with triangle-based reserving workflows. It is designed to be seamlessly integrated with existing reserving and pricing tools.

1.5 Audience

This manual is written for:

- Practising actuaries building loss models in Python.
- Researchers who require reproducible synthetic claim data for methodology development.
- Students and educators who want to demonstrate stochastic actuarial techniques with transparent code.
- Open-source contributors and reviewers who wish to inspect or extend the package.

1.6 How to Read This Manual

The manual is organised so that new users can follow it from the installation chapter through quick start and core concepts before exploring the main classes. Experienced users can jump directly to the API reference or worked examples. The validation chapter is intended to support peer review and open-source scrutiny.

Chapter 2

Installation

2.1 Requirements

ActSim requires Python 3.8 or later. The package depends on a number of common scientific Python libraries:

- `numpy` (≥ 1.26)
- `scipy` (≥ 1.7)
- `pandas` (≥ 1.3)
- `matplotlib` (≥ 3.4)
- `seaborn` (≥ 0.13)
- `actstats` ($\geq 0.1.1$)

These packages will be installed automatically when ActSim is installed via `pip`.

2.2 Standard Installation

ActSim can be installed from the Python Package Index using `pip`:

```
pip install actsim
```

This will install the latest stable release together with all required dependencies.

2.3 Installation from Source

To install the latest development version directly from the source repository, clone the repository and install in editable mode:

```
git clone https://github.com/casact/actsim.git
cd actsim
pip install -e .
```

Editable installation is convenient when modifying the package locally, since changes to the source files take effect without reinstallation.

2.4 Development Installation

Contributors who plan to add features, fix bugs, or run the test suite should install the additional development dependencies:

```
git clone https://github.com/casact/actsim.git
cd actsim
pip install -e .[dev]
```

The development extras include testing utilities such as `pytest`, formatters, and other tools used in the contribution workflow.

2.5 Recommended Virtual Environment

It is recommended to install ActSim into an isolated virtual environment to avoid conflicts with other Python projects:

```
python -m venv venv
source venv/bin/activate      # macOS / Linux
venv\Scripts\activate        # Windows
pip install actsim
```

2.6 Verifying the Installation

After installation, verify that ActSim can be imported and that the default configuration loads successfully:

```
from actsim import (
    DistributionFitter,
    StochasticSimulator,
    ClaimSimulator,
    load_config,
)

config = load_config()
print("Severity distributions:", config.distributions["severity"])
print("Frequency distributions:", config.distributions["frequency"])
print("Metrics:", config.metrics)
```

If the import statements succeed and the configuration prints the expected lists of distributions and metrics, the installation is working correctly.

Chapter 3

Quick Start

This chapter is intended to get a new user running their first simulation within a few minutes. The example below illustrates a basic frequency–severity simulation using ActSim’s `StochasticSimulator`.

3.1 Minimal Example

The following script simulates 10,000 years of aggregate losses, using a Poisson frequency distribution with mean 50 and a lognormal severity distribution.

```
from actsim import StochasticSimulator

simulator = StochasticSimulator(
    freq_dist="poisson",
    freq_params=(50,),
    sev_dist="lognormal",
    sev_params=(10, 0.5),
    num_sim=10000,
    keep_all=True,
    seed=1234,
)

# Run the simulation
simulator.gen_agg_simulations()

# Inspect the simulated aggregate losses
print(simulator.results.head())

# Calculate the 99th percentile of the aggregate loss distribution
var_99 = simulator.calc_agg_percentile(99)
print(f"99% VaR: {var_99:,.2f}")
```

3.2 What the Example Does

The example performs the following steps:

1. Constructs a `StochasticSimulator` configured with a Poisson frequency distribution and a lognormal severity distribution.
2. Sets the number of simulated years to 10,000.
3. Sets `keep_all=True` so that individual event-level data are stored alongside the aggregate results, enabling later calculation of metrics such as Occurrence Exceedance Probability.
4. Fixes the random seed to 1234 to make the simulation reproducible.
5. Calls `gen_agg_simulations()` to run the simulation.
6. Calculates the 99th percentile of the aggregate loss distribution.

3.3 Next Steps

After running this minimal example, users may wish to:

- Explore the `DistributionFitter` class for fitting distributions to observed loss data (Chapter 6).
- Use the `ClaimSimulator` class to generate synthetic claim-level data from a policy table (Chapter 8).
- Apply deductibles, limits, and other contract terms to the simulated losses (Section 7.7).
- Integrate the simulated data with the `chainladder` package for reserving analysis (Chapter 10).

The remainder of the manual covers each capability in detail, with worked examples and validation guidance.

Chapter 4

Core Concepts

This chapter introduces the actuarial concepts that underpin ActSim. Users familiar with frequency–severity modelling and stochastic simulation may skim this material, but the notation introduced here is used throughout the manual.

4.1 Frequency Distributions

The *frequency* of claims is the number of claims that occur during a specified period (often a policy year). ActSim models claim frequency using discrete probability distributions. The most common choices are:

- **Poisson distribution.** A simple one-parameter distribution where the mean equals the variance. The probability mass function is

$$P(N = n) = \frac{\lambda^n e^{-\lambda}}{n!}, \quad n = 0, 1, 2, \dots$$

where $\lambda > 0$ is the expected number of claims.

- **Negative binomial distribution.** A two-parameter distribution that allows variance greater than the mean (over-dispersion), often more realistic for claim counts.

4.2 Severity Distributions

The *severity* of a claim is the loss amount associated with that claim. ActSim supports a wide range of continuous severity distributions, including:

- Lognormal
- Gamma
- Weibull
- Pareto
- Exponential
- Beta

- Normal, logistic, uniform (occasionally used for sensitivity analysis or stylised examples)

For a continuous severity X with density f_X and cumulative distribution function F_X , the conventional moments are

$$E[X] = \int_0^\infty x f_X(x) dx, \quad \text{Var}(X) = E[X^2] - (E[X])^2.$$

4.3 Aggregate Loss

The *aggregate loss* for a period is the total loss across all claims that occur during that period. If N denotes the number of claims and $X_1, X_2, \dots, X_N$ denote the individual claim severities, then the aggregate loss is

$$S = \sum_{i=1}^N X_i.$$

When N and the X_i are mutually independent and the X_i are independent and identically distributed, S is referred to as a *compound distribution*. Standard moment results in this setting are

$$E[S] = E[N] \cdot E[X], \quad \text{Var}(S) = E[N] \cdot \text{Var}(X) + \text{Var}(N) \cdot (E[X])^2.$$

In ActSim, aggregate losses are typically estimated by simulation rather than by analytical closed-form expressions, which makes the framework flexible across distribution choices.

4.4 Dependency and Correlation

In practice, claim frequency and severity are not always independent. For example, a year with many claims may also feature larger average severities due to a common driver. ActSim supports two mechanisms for introducing dependency between frequency and severity:

1. **Linear correlation.** A user-supplied correlation coefficient is applied to a bivariate normal copula structure via Cholesky decomposition.
2. **Copula-based dependency.** ActSim supports several standard copulas, including Gaussian, Frank, Gumbel, and Clayton copulas, parametrised by a dependency parameter.

Multivariate dependency across multiple lines of business is also supported through correlated marginal simulation, configured via a correlation matrix and a list of marginal distributions.

4.5 Claim-Level Simulation

Many actuarial workflows require not just aggregate losses but also *claim-level* data: individual claim records with policy identifiers, occurrence dates, severities, and development histories. ActSim's `ClaimSimulator` class generates these records by:

1. Grouping policies by their frequency and severity distribution specifications.
2. Simulating claim counts and severities for each group using the underlying `StochasticSimulator`.

3. Assigning occurrence dates within each policy's exposure window using a non-homogeneous Poisson process.
4. Optionally applying claim development to each simulated claim.

4.6 Claim Development

Claims do not typically reach their ultimate cost on the date they occur. Incurred and paid amounts develop over time as additional information becomes available. ActSim represents this development through user-supplied *loss development factors* (LDFs).

If U denotes the ultimate loss for a claim and CDF_a denotes the cumulative development factor to ultimate at age a , then the incurred loss at age a is approximated by

$$L_a = \frac{U}{CDF_a}.$$

The incremental development between two adjacent ages is

$$\Delta L_a = L_a - L_{a-1}.$$

Stochastic variation around expected LDFs is introduced via a normal multiplier with user-specified volatility.

4.7 Risk Metrics

ActSim provides several standard risk metrics, including:

- **Mean and standard deviation** of aggregate losses.
- **Percentiles** of the loss distribution.
- **Value at Risk (VaR)** at user-specified confidence levels.
- **Tail Value at Risk (TVaR)**, also known as conditional tail expectation.
- **Aggregate Exceedance Probability (AEP)**, related to aggregate (annual) losses.
- **Occurrence Exceedance Probability (OEP)**, related to maximum single-event losses within a year.

These metrics are defined formally in Chapter 9.

Chapter 5

ActStats

5.1 Overview

ActSim depends on the companion package `actstats`, authored by Juntao Zhang and Tingting Shi, which provides actuarial-style wrappers around standard statistical distributions. The `actstats` package exposes a familiar interface for fitting, sampling, and evaluating distributions, and it is the engine behind ActSim’s distribution fitting and stochastic simulation classes.

The `actstats` package is imported under the alias `act` throughout the manual:

```
from actstats import actuarial as act
```

5.2 Why a Dedicated Actuarial Wrapper?

Many actuarial computations could be performed directly using `scipy.stats` or `numpy.random`. However, parameter conventions in `scipy` and `numpy.random` do not always match standard actuarial textbook conventions. For example, `scipy.stats.lognorm` is parametrised by a shape parameter s and optional location and scale arguments, whereas actuaries typically work with the underlying normal mean μ and standard deviation σ .

`actstats` provides actuarial-friendly parametrisations and exposes a consistent API across distributions. It also provides specialised utilities such as the non-homogeneous Poisson process used for incurred date simulation in ActSim.

5.3 Supported Distributions

The distributions exposed through `actstats` that are used internally by ActSim include:

- Continuous: `normal`, `logistic`, `exponential`, `gamma`, `beta`, `lognormal`, `weibull`, `pareto`, `uniform`.
- Discrete: `poisson`, `negative_binomial`.

These names match the entries used in the default `config.yaml` shipped with ActSim and may be referenced from the configuration loader.

5.4 Parameter Conventions

The parameter conventions are described in [Section 17.2](#)

5.5 Common Operations

The interface exposed by `actstats` mirrors familiar statistical patterns:

```
from actstats import actuarial as act

# Construct a frozen distribution
dist = act.lognormal(0.5, 0.2)

# Draw random samples
samples = dist.rvs(size=10000)

# Evaluate the probability density function
pdf_values = dist.pdf([1.0, 1.5, 2.0])

# Evaluate the cumulative distribution function
cdf_values = dist.cdf([1.0, 1.5, 2.0])

# Evaluate the inverse CDF (quantile function)
quantiles = dist.ppf([0.5, 0.9, 0.99])
```

5.6 Discrete Distributions

Frequency simulations rely on discrete distributions such as Poisson and negative binomial. Sampling is performed in the same way:

```
from actstats import actuarial as act

# Poisson with mean 10
freq_dist = act.poisson(10)
counts = freq_dist.rvs(size=1000)

# 80th percentile of Poisson(10)
q80 = act.poisson.ppf(0.8, 10)
```

5.7 Non-Homogeneous Poisson Process

`actstats` also provides a utility for simulating event times from a non-homogeneous Poisson process (NHPP) with a sinusoidal seasonal intensity:

```
from actstats import actuarial as act

# Baseline intensity 10, seasonality amplitude 0.5,
# phase shift 0, exposure 1 year
nhpp = act.nonhomogeneous_poisson(10, 0.5, 0, 1)
```

```
# Generate event time fractions in [0, 1]  
times = nhpp.rvs(n_events=20)
```

This utility is used internally by `ClaimSimulator` to assign incurred dates to simulated claims (see Chapter 8).

5.8 Date Conversion Helpers

`actstats` also provides date conversion helpers, such as `fraction_to_date_full`, that map fractions of an exposure period to calendar dates. These are used by ActSim when assigning incurred and development dates to simulated claims.

Chapter 6

DistributionFitter

6.1 Purpose

The `DistributionFitter` class is used to fit one or more candidate probability distributions to an observed dataset and to compare them using goodness-of-fit metrics. In actuarial practice, this step typically precedes simulation: the analyst inspects historical claim counts or severities, selects a plausible distributional form, and validates the choice using diagnostic statistics.

6.2 Supported Distributions

`DistributionFitter` supports the same distributions exposed by `actstats`, summarised below.

Type	Distributions
Continuous severity	normal, logistic, exponential, gamma, beta, lognormal, weibull, pareto, uniform
Discrete frequency	poisson, negative binomial

The distributions used in any particular fitting run can be controlled through the `distributions` argument or via a YAML configuration file.

6.3 Goodness-of-Fit Metrics

For each fitted distribution, `DistributionFitter` computes:

- Log-likelihood
- Akaike Information Criterion (AIC)
- Bayesian Information Criterion (BIC)
- Chi-square statistic
- Kolmogorov–Smirnov statistic

The information criteria are defined as

$$\text{AIC} = 2k - 2 \ln L, \quad \text{BIC} = k \ln n - 2 \ln L,$$

where k is the number of estimated parameters, n is the sample size, and L is the likelihood evaluated at the maximum-likelihood estimate.

By default, the metrics used for ranking are AIC and BIC. Additional metrics can be requested by passing a custom `metrics` list.

6.4 Basic Workflow

The typical workflow uses four steps:

1. Load configuration and select candidate distributions.
2. Construct a `DistributionFitter` with the data.
3. Call `fit()` to estimate parameters and compute metrics.
4. Review and select the best-fit distribution.

```
from actsim import DistributionFitter, load_config
from actstats import actuarial as act

# Generate example severity data
sev_data = act.lognormal(0.5, 0.2).rvs(size=10000)

# Load configuration
config = load_config()
distributions = config.distributions["severity"]
metrics = config.metrics

# Construct the fitter
fitter = DistributionFitter(
    sev_data,
    distributions=distributions,
    metrics=metrics,
)

# Fit candidate distributions
fitter.fit()
```

6.5 Inspecting Results

After fitting, the best-fit distribution under each metric is stored in the `best_fits` attribute, while the currently selected distribution is stored in `selected_fit`.

```
print("Best fits:", fitter.best_fits)
print("Selected fit:", fitter.selected_fit)
print("Selected distribution:", fitter.get_selected_dist())
print("Selected parameters:", fitter.get_selected_params())
```

6.6 Manually Selecting a Distribution

Users can override the automatic selection by name:

```
fitter.select_distribution("gamma")
print(fitter.selected_fit["params"])
```

This is useful when the analyst has a prior reason to prefer a particular distribution, or when comparing scenarios.

6.7 Sampling and Prediction

Once a distribution is selected, the fitter can be used as a drop-in sampler. The `sample` method returns random draws from the selected distribution, while `sample_mixed` inserts a proportion of zeros and ones to handle datasets with mass at those points (e.g. zero-inflated severity data).

```
# Draw 10 samples
samples = fitter.sample(size=10)

# Draw 100 samples with 10% zeros and 5% ones
mixed = fitter.sample_mixed(zero_prop=0.1, one_prop=0.05, size=100)

# Evaluate the predicted density at given points
density = fitter.predict([0.5, 1.0, 1.5, 2.0])
```

6.8 Diagnostic Plots

The `plot_predictions()` method draws a histogram of the data together with the probability density functions of the fitted candidate distributions. This plot is the primary visual diagnostic for assessing fit quality.

```
fitter.plot_predictions()
```

6.9 Summary Reports

Two methods produce numerical summaries:

- `calculate_statistics()` returns a comparison of the mean, standard deviation, and key percentiles between the observed data and the predicted density.
- `summary()` returns a DataFrame of all fitted distributions, their parameters, and their goodness-of-fit metrics.

```
fitter.calculate_statistics()
fitter.summary()
```

6.10 Practical Notes

A few practical considerations apply when using `DistributionFitter`:

- Some distributions (e.g. Pareto, beta) require positive or bounded support. If the input data violate the support, the fit for that distribution may fail; `DistributionFitter` catches such failures and continues with the remaining distributions.
- AIC and BIC are convenient ranking tools but should be interpreted alongside diagnostic plots, especially in the tails, which often matter most for actuarial applications.
- For frequency data, choose distributions appropriate to count data (e.g. Poisson, negative binomial) rather than continuous distributions.

Chapter 7

StochasticSimulator

7.1 Purpose

The `StochasticSimulator` class is the core simulation engine of `ActSim`. It supports Monte Carlo simulation of aggregate losses under specified frequency and severity distributions and can also be used to generate correlated multivariate simulations across multiple lines of business.

The simulator is designed to be reproducible: every run is governed by an explicit random seed, and intermediate event-level data can be retained for downstream metrics such as Occurrence Exceedance Probability.

7.2 Construction

A simulator is constructed by specifying the frequency and severity distributions together with their parameters:

```
from actsim import StochasticSimulator

simulator = StochasticSimulator(
    freq_dist="poisson",
    freq_params=(10,),
    sev_dist="lognormal",
    sev_params=(10, 0.5),
    num_sim=10000,
    keep_all=True,
    seed=1234,
)
```

The constructor arguments are summarised below.

Argument	Description
<code>freq_dist</code>	Name of the frequency distribution.
<code>freq_params</code>	Tuple of parameters for the frequency distribution.
<code>sev_dist</code>	Name of the severity distribution.
<code>sev_params</code>	Tuple of parameters for the severity distribution.
<code>num_sim</code>	Number of simulation periods (default 10,000).
<code>keep_all</code>	If <code>True</code> , retain individual event records.
<code>seed</code>	Random seed for reproducibility.
<code>correlation</code>	Optional linear correlation between N and X .
<code>copula_type</code>	Optional copula family.
<code>theta</code>	Copula dependence parameter.

7.3 Frequency–Severity Simulation

The primary method is `gen_agg_simulations()`, which loops over the requested number of simulation periods, draws a frequency from the frequency distribution, and then draws that many severities from the severity distribution. Aggregate losses are accumulated and stored in `results`.

```
simulator.gen_agg_simulations()
print(simulator.results.head())
```

If `keep_all=True`, individual event-level records are stored in `all_simulations`, with columns `year`, `event_id`, `yearly_event_id`, and `amount`.

7.4 Dependency between Frequency and Severity

7.4.1 Linear Correlation

When `correlation` is supplied without a copula, frequency and severity are correlated through a Cholesky factorisation of a 2×2 correlation matrix:

$$C = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}, \quad C = LL^\top.$$

Standard normal variates are transformed into uniform variates via the normal CDF, and these uniforms are then mapped through the inverse CDF of the target frequency and severity distributions.

```
simulator = StochasticSimulator(
    "poisson", (10,),
    "lognormal", (10, 0.5),
    num_sim=10000,
    keep_all=True,
    seed=1234,
    correlation=0.6,
)
```

7.4.2 Copula-Based Dependency

ActSim supports four copula families: Gaussian, Frank, Gumbel, and Clayton. Each is parametrised by a dependence parameter θ (or by a correlation matrix in the Gaussian case).

```
simulator = StochasticSimulator(
    "poisson", (10,),
    "lognormal", (10, 0.5),
    num_sim=10000,
    keep_all=True,
    seed=1234,
    correlation=0.6,
    copula_type="frank",
    theta=0.6,
)
```

The chosen copula generates pairs of dependent uniforms, which are then mapped to frequency and severity via the inverse CDFs.

7.5 Multivariate Correlated Simulation

The method `gen_multivariate_corr_simulations` generates correlated losses across multiple lines of business. It accepts a correlation matrix as a CSV file and a list of marginal distributions as a JSON file.

```
simulator = StochasticSimulator(
    "normal", (1, 0),
    "normal", (1, 0),
    num_sim=100000,
    keep_all=True,
    seed=1234,
)

simulator.gen_multivariate_corr_simulations(
    "examples/correlated_sim/corr_matrix.csv",
    "examples/correlated_sim/dist_list.json",
    gen_marginal=True,
)
```

When `gen_marginal=True`, the simulator stores marginal simulations for each line of business in `_all_simulations_data`. The aggregated total across all lines is stored in `results`.

7.6 Analysing Results

7.6.1 Aggregate Percentiles

The `calc_agg_percentile` method returns a single percentile of the aggregate loss distribution.

```
p99_2 = simulator.calc_agg_percentile(99.2)
print(f"99.2% percentile: {p99_2:,.2f}")
```

7.6.2 Risk Metrics

The `analyze_results` method computes a table of VaR, TVaR, OEP, and AEP at user-specified quantiles. OEP is only available when `keep_all=True`.

```
metrics = simulator.analyze_results(  
    quantiles=[0.5, 0.75, 0.9, 0.95, 0.99]  
)  
print(metrics)
```

7.6.3 Distribution Plot

The `plot_distribution` method draws a histogram of the simulated aggregate losses, optionally on a logarithmic scale:

```
simulator.plot_distribution(bins=100, log_option=False)
```

7.6.4 Correlation Plot

If `keep_all=True`, the `plot_correlated_variables` method draws a scatter and density plot of yearly event counts against mean severity, displaying the empirical Pearson correlation. This plot is useful for sanity-checking whether the imposed dependency structure behaves as expected.

```
simulator.plot_correlated_variables()
```

7.7 Applying Deductibles and Limits

Insurance contracts often include per-occurrence and aggregate deductibles and limits. These can be applied to simulated losses with `apply_deductible_and_limit`:

```
gross_loss = simulator.apply_deductible_and_limit(  
    per_occurrence_ded=1000,  
    per_occurrence_limit=10000,  
    agg_ded=100000,  
    agg_limit=300000,  
)
```

The method first reduces each event's loss by the per-occurrence deductible (with a floor at zero), caps the result at the per-occurrence limit, then aggregates by year and applies the annual aggregate deductible and limit.

The returned DataFrame contains the gross (capped and floored) annual losses. To run further analyses on the post-contract losses, supply the DataFrame as the `all_simulations` keyword argument to `analyze_results`:

```
gross_loss["amount"] = gross_loss["gross_loss"]  
metrics = simulator.analyze_results(all_simulations=gross_loss)
```

7.8 Reproducibility

All random draws inside `StochasticSimulator` are governed by the `seed` argument supplied at construction. Setting the seed to a fixed value guarantees that the same input produces the same output across runs, which is essential for validation and audit. The recommended practice is to record the seed alongside any results that are reported externally.

Chapter 8

ClaimSimulator

8.1 Purpose

The `ClaimSimulator` class generates synthetic claim-level data from a policy table. While `StochasticSimulator` produces aggregate losses for a single distributional specification, `ClaimSimulator` is designed for portfolios of policies that may have different exposure windows and different frequency or severity assumptions.

This is the right tool when the analyst needs:

- A claim-level dataset with policy identifiers and occurrence dates.
- Synthetic data suitable for testing reserving methods.
- A starting point for downstream development analysis (e.g. loss triangles).

8.2 Required Policy Schema

The class accepts a `pandas.DataFrame` with one row per policy. The required columns are listed below.

Column	Description
<code>policy_id</code>	Unique identifier for the policy.
<code>freq_dist</code>	Name of the frequency distribution.
<code>freq_params</code>	Tuple of frequency distribution parameters.
<code>sev_dist</code>	Name of the severity distribution.
<code>sev_params</code>	Tuple of severity distribution parameters.
<code>start_date</code>	Policy effective date.
<code>end_date</code>	Policy expiration date.

The constructor validates that all required columns are present and that `freq_params` and `sev_params` are tuples.

8.3 Construction

A typical policy table and `ClaimSimulator` construction looks like this:

```
import pandas as pd
import numpy as np
from actsim import ClaimSimulator

policies = pd.DataFrame({
    "policy_id": range(1, 101),
    "freq_dist": "poisson",
    "freq_params": list(zip(np.random.uniform(0.6, 0.8, 100).round(2),)),
    "sev_dist": "lognormal",
    "sev_params": list(zip(
        np.random.uniform(8, 12, 100).round(2),
        np.random.uniform(0.3, 0.7, 100).round(2),
    )),
    "start_date": pd.Timestamp("2023-01-01"),
    "end_date": pd.Timestamp("2023-12-31"),
})

claim_sim = ClaimSimulator(policies, random_seed=42)
```

Optional arguments include `correlation`, `copula_type`, and `copula_param` for introducing dependency between frequency and severity within each policy group.

8.4 Simulating Claims

The `simulate_claims` method groups policies by their distribution specifications, runs a separate `StochasticSimulator` for each group, and assembles the results into a unified claim `DataFrame`.

```
claim_sim.simulate_claims()
print(claim_sim.claim_data.head())
```

The resulting `claim_data` `DataFrame` contains the columns `year`, `event_id`, `yearly_event_id`, `amount`, `policy_id`, `start_date`, and `end_date`. The `year` column corresponds to the policy index within its group rather than a calendar year, and is used internally to map simulated claims back to their parent policy. This design reflects the package's original focus on catastrophe modelling, and the column naming and structure will be refined in a future version.

8.4.1 Policies with Zero Simulated Claims

It is possible for a policy to produce zero simulated claims, especially when the expected frequency is below one. In that case, the policy remains in the `policies` table but does not appear in `claim_data`. ActSim does not, by default, create explicit zero-claim records. If downstream analyses require a record per policy regardless of claim activity, the user should join `claim_data` back onto `policies` and treat missing rows as zero.

8.5 Simulating Claim Dates

After claims have been simulated, occurrence dates can be assigned using a non-homogeneous Poisson process via the `simulate_dates_nhpp` method.

```
claim_sim.simulate_dates_nhpp(
```

```

    lambda0=10,
    alpha=0.5,
    phase=0,
    T=1,
)

```

The arguments control the seasonal intensity:

- `lambda0` – baseline intensity.
- `alpha` – seasonality amplitude.
- `phase` – phase shift of the seasonality.
- `T` – duration of the exposure period in years.

For each claim, the method draws a fraction of the exposure period from the NHPP and maps that fraction to a date within the relevant policy’s active window. The amount column is renamed to `ultimate_loss` in preparation for the development step.

8.6 Simulating Claim Development

The `simulate_claim_development` method develops each claim backwards from its ultimate amount, using a user-supplied set of base loss development factors and a stochastic volatility multiplier.

```

base_LDFs = {
    0: 2.00,
    3: 1.50,
    6: 1.20,
    9: 1.10,
    12: 1.05,
    15: 1.02,
    18: 1.00,
}

claim_sim.simulate_claim_development(
    base_LDFs=base_LDFs,
    volatility=0.1,
    cumulative_factor=1.0,
)

```

For each claim, a unique set of stochastic LDFs is drawn by multiplying each base factor by a normal random variable with mean 1 and standard deviation `volatility`. Cumulative development factors (CDFs) are constructed by chaining the stochastic LDFs from the latest age backwards. The incurred loss at age a is then computed as

$$L_a = \frac{U}{\text{CDF}_a},$$

where U is the ultimate loss for the claim.

8.6.1 Output Schema

The `claim_development` attribute is a long-format `DataFrame` with one row per (claim, development month). The columns are listed below.

Column	Description
<code>accident_year</code>	Accident year of the claim's incurred date.
<code>incurred_date</code>	Date the claim occurred.
<code>claim_id</code>	Unique claim identifier.
<code>policy_id</code>	Identifier of the parent policy.
<code>development_month</code>	Development age in months.
<code>incurred_loss</code>	Incurred loss at this age.
<code>development_date</code>	Calendar date corresponding to the development age.

8.7 Saving Claim Development Data

Simulated development data can be persisted to CSV using the `save_claim_development` method:

```
claim_sim.save_claim_development("output/claim_development.csv")
```

The resulting file is suitable for ingestion by triangle-based reserving tools, including the `chainladder` package.

Chapter 9

Risk Metrics

9.1 Overview

Once a simulation has been run, the analyst typically wants to summarise the loss distribution using a small number of risk metrics. `ActSim` provides functions to compute the most widely used metrics directly from simulated data.

This chapter defines each metric formally and describes how to obtain it from the `StochasticSimulator` class.

9.2 Mean and Standard Deviation

The mean and standard deviation of the simulated aggregate losses are the simplest summary statistics:

$$\bar{S} = \frac{1}{M} \sum_{m=1}^M S_m, \quad \hat{\sigma}_S = \sqrt{\frac{1}{M-1} \sum_{m=1}^M (S_m - \bar{S})^2},$$

where S_m is the aggregate loss in simulation m and M is the number of simulations. Both can be obtained directly from the `results` attribute of the simulator:

```
mean_loss = simulator.results.mean()
std_loss  = simulator.results.std()
```

9.3 Percentiles

A percentile of the simulated aggregate losses is computed using `calc_agg_percentile`:

```
p95 = simulator.calc_agg_percentile(95)
p99 = simulator.calc_agg_percentile(99)
```

9.4 Value at Risk (VaR)

The Value at Risk at confidence level $q \in (0, 1)$ is defined as the q -th quantile of the aggregate loss distribution:

$$\text{VaR}_q(S) = \inf\{s : F_S(s) \geq q\},$$

where F_S is the cumulative distribution function of S . In a simulation context, VaR_q is estimated by the empirical q -quantile of the simulated values. The `analyze_results` method returns VaR at user-specified quantiles:

```
metrics = simulator.analyze_results(quantiles=[0.95, 0.99])
print(metrics["VaR"])
```

9.5 Tail Value at Risk (TVaR)

The Tail Value at Risk, also known as the conditional tail expectation or expected shortfall, is the expected loss conditional on exceeding the corresponding VaR threshold:

$$\text{TVaR}_q(S) = \mathbb{E}[S \mid S > \text{VaR}_q(S)].$$

In simulation, TVaR_q is estimated by averaging the simulated values that exceed the empirical VaR_q .

```
metrics = simulator.analyze_results(quantiles=[0.95, 0.99])
print(metrics["TVaR"])
```

TVaR is a coherent risk measure and is preferred over VaR for applications where the behaviour of the loss distribution beyond the VaR threshold is material.

9.6 Aggregate Exceedance Probability (AEP)

The Aggregate Exceedance Probability at level q is the q -th percentile of the *aggregate* (annual) loss distribution:

$$\text{AEP}_q = \inf\{s : F_S(s) \geq q\}.$$

For a single-period simulation, AEP and VaR coincide. AEP is widely reported in catastrophe risk and reinsurance contexts as a function of return period, where return period T corresponds to confidence level $q = 1 - 1/T$.

```
metrics = simulator.analyze_results(quantiles=[0.95, 0.99])
print(metrics["aep"])
```

9.7 Occurrence Exceedance Probability (OEP)

The Occurrence Exceedance Probability at level q is the q -th percentile of the *maximum single-event* loss within a year. If M is the maximum single-event loss in a year,

$$\text{OEP}_q = \inf\{m : F_M(m) \geq q\}.$$

OEP requires event-level data and therefore can only be computed when the simulator is run with `keep_all=True`. In that case `analyze_results` returns OEP alongside the other metrics:

```
simulator = StochasticSimulator(  
    "poisson", (10,),  
    "lognormal", (10, 0.5),  
    num_sim=10000,  
    keep_all=True,  
    seed=1234,  
)  
simulator.gen_agg_simulations()  
metrics = simulator.analyze_results(quantiles=[0.95, 0.99])  
print(metrics["oep"])
```

9.8 Putting it Together

A typical risk-metrics report contains VaR, TVaR, AEP, and OEP at a small set of confidence levels. The example below produces such a report and prints it as a single DataFrame.

```
quantiles = [0.5, 0.75, 0.9, 0.95, 0.99, 0.995]  
report = simulator.analyze_results(quantiles=quantiles)  
print(report.round(2))
```

The output table contains one row per quantile and one column per metric. This format is convenient for inclusion in technical memos and audit packages.

9.9 Choice of Metric

Different metrics emphasise different aspects of the loss distribution:

- **VaR** is simple and widely understood, but does not describe behaviour beyond the threshold.
- **TVaR** captures tail behaviour and is preferred in frameworks that require coherent risk measures.
- **AEP and OEP** are particularly common in property-catastrophe and reinsurance contexts, where annual aggregate exposure and the largest single event need to be distinguished.

The choice of metric should be driven by the regulatory framework, the contract structure, and the question being asked. ActSim makes all of these metrics available from a single simulation run, allowing the analyst to report whichever combination is most relevant.

Chapter 10

Integration with Chainladder

10.1 Overview

ActSim is designed to be seamlessly integrated with the established triangle-based reserving tools. The `chainladder` Python package provides a comprehensive set of methods for reserving, including chain-ladder development, Bornhuetter–Ferguson, Cape Cod, Mack, bootstrap chain-ladder, and various GLM-based approaches.

A natural division of labour is for ActSim to generate synthetic claim-level data and convert the results into triangle format, while `chainladder` performs reserving analysis on the resulting triangle. This chapter shows how the two packages interact.

10.2 From Simulated Claims to a Triangle

Once `ClaimSimulator` has produced a long-format development `DataFrame`, the data can be reshaped into the format expected by `chainladder`.

```
import pandas as pd
import numpy as np
from actsim import ClaimSimulator

# Build a policy table and simulate claims
policies = pd.DataFrame({
    "policy_id": range(1, 501),
    "freq_dist": "poisson",
    "freq_params": list(zip(np.random.uniform(0.6, 0.8, 500).round(2),)),
    "sev_dist": "lognormal",
    "sev_params": list(zip(
        np.random.uniform(8, 12, 500).round(2),
        np.random.uniform(0.3, 0.7, 500).round(2),
    )),
    "start_date": pd.Timestamp("2018-01-01"),
    "end_date": pd.Timestamp("2022-12-31"),
})

claim_sim = ClaimSimulator(policies, random_seed=42)
claim_sim.simulate_claims()
claim_sim.simulate_dates_nhpp(lambda0=10, alpha=0.5, phase=0, T=5)
```

```

base_LDFs = {0: 2.0, 3: 1.5, 6: 1.2, 9: 1.1, 12: 1.05, 15: 1.02, 18: 1.0}
claim_sim.simulate_claim_development(
    base_LDFs=base_LDFs,
    volatility=0.1,
    tail_factor=1.0,
)

triangle_df = (
    claim_sim.claim_development
    .groupby(["accident_year", "development_month"])["incurred_loss"]
    .sum()
    .reset_index()
)

```

The resulting `triangle_df` has one row per (accident year, development age) cell and aggregates the incurred losses across all claims in that cell. Since the `chainladder` package can also work directly with claim-level loss data and perform the aggregation internally, this preprocessing step is optional.

10.3 Constructing a chainladder Triangle

The `chainladder` package accepts long-format data and converts it into its native triangle representation:

```

import chainladder as cl

triangle = cl.Triangle(
    triangle_df,
    origin="accident_year",
    development="development_month",
    columns="incurred_loss",
    cumulative=True,
)

print(triangle)

```

The `cumulative=True` flag indicates that the values represent cumulative incurred losses; if instead the values represented incremental amounts, the flag should be set to `False`.

10.4 Position of ActSim Relative to Chainladder

ActSim is positioned as a simulation and data-generation companion to `chainladder` for analytical and scenario testing purposes. In a typical workflow:

- ActSim generates synthetic policy and claim-level data.
- ActSim develops the simulated claims through a user-supplied LDF schedule.
- The resulting development data are reshaped into a triangle.
- `chainladder` performs reserving analyses on the triangle.

- The reserving outputs are compared with the underlying simulated ultimates to validate the workflow.

This division of labour leverages the strengths of each package: **ActSim**'s flexible, transparent simulation engine and **chainladder**'s comprehensive suite of reserving methods.

Chapter 11

Worked Examples

This chapter contains a series of end-to-end worked examples designed to illustrate typical ActSim workflows. Each example states its purpose, describes the input data, presents the Python code, and discusses the output and its actuarial interpretation.

11.1 Example 1: Aggregate Loss Simulation

Purpose. Simulate aggregate losses for a single line of business using a Poisson frequency distribution and a lognormal severity distribution.

Input data. No external data; the example uses synthetic distribution parameters.

Code.

```
from actsim import StochasticSimulator

simulator = StochasticSimulator(
    freq_dist="poisson",
    freq_params=(50,),
    sev_dist="lognormal",
    sev_params=(10, 0.5),
    num_sim=50000,
    keep_all=True,
    seed=2024,
)

simulator.gen_agg_simulations()

print("Mean aggregate loss:", simulator.results.mean())
print("99% VaR:", simulator.calc_agg_percentile(99))
print("99.5% VaR:", simulator.calc_agg_percentile(99.5))
```

Output interpretation. The mean aggregate loss should be close to $E[N] \cdot E[X]$. Because the severity is lognormal with parameters $\mu = 10$ and $\sigma = 0.5$, $E[X] \approx \exp(\mu + \sigma^2/2)$. The simulated mean should converge to this theoretical mean as the number of simulations increases.

Actuarial notes. This example serves as a sanity check that the simulator correctly implements the compound Poisson–lognormal model. The 99% VaR and 99.5% VaR are typical risk indicators reported in solvency frameworks.

11.2 Example 2: Fitting Severity Distributions to Historical Losses

Purpose. Fit a set of candidate severity distributions to a synthetic dataset and select the best-fitting distribution by AIC.

Input data. A simulated dataset of 10,000 losses drawn from a known lognormal distribution. In a real engagement, this dataset would be replaced with empirical losses from claim files.

Code.

```
from actsim import DistributionFitter, load_config
from actstats import actuarial as act

# Generate synthetic loss data
loss_data = act.lognormal(0.5, 0.2).rvs(size=10000)

# Load the default configuration
config = load_config()
distributions = config.distributions["severity"]
metrics = config.metrics

# Fit candidate distributions
fitter = DistributionFitter(loss_data,
                           distributions=distributions,
                           metrics=metrics)

fitter.fit()

# Inspect best fits and selected distribution
print("Best fit by AIC:", fitter.best_fits["aic"]["name"])
print("Selected fit:", fitter.selected_fit["name"])

# Diagnostic plot
fitter.plot_predictions()

# Summary table
print(fitter.summary())
```

Output interpretation. The selected distribution should typically be the lognormal, since the data were drawn from that family. The summary table reports AIC, BIC, log-likelihood, and chi-square values for each fitted distribution.

Actuarial notes. In real-world fits, the differences between candidate distributions in AIC and BIC are often small. Diagnostic plots and tail-fit checks should be consulted in addition to the information criteria.

11.3 Example 3: Synthetic Policy and Claim Generation

Purpose. Generate a synthetic dataset of 100 policies and their associated claims.

Input data. A randomly generated policy table.

Code.

```
import pandas as pd
import numpy as np
from actsim import ClaimSimulator

policies = pd.DataFrame({
    "policy_id": range(1, 101),
    "freq_dist": "poisson",
    "freq_params": list(zip(np.random.uniform(0.6, 0.8, 100).round(2),)),
    "sev_dist": "lognormal",
    "sev_params": list(zip(
        np.random.uniform(8, 12, 100).round(2),
        np.random.uniform(0.3, 0.7, 100).round(2),
    )),
    "start_date": pd.Timestamp("2023-01-01"),
    "end_date": pd.Timestamp("2023-12-31"),
})

claim_sim = ClaimSimulator(policies, random_seed=42)
claim_sim.simulate_claims()
print(claim_sim.claim_data.head())
```

Output interpretation. The `claim_data` DataFrame contains one row per simulated claim, with a `policy_id` column linking each claim back to its parent policy.

Actuarial notes. Some policies may produce zero claims; this is expected behaviour for low-frequency policies and is consistent with the underlying Poisson assumption.

11.4 Example 4: Developing Claims Using LDFs

Purpose. Develop the simulated claims from Example 3 using a user-supplied LDF schedule.

Code.

```
claim_sim.simulate_dates_nhpp(lambda0=10, alpha=0.5, phase=0, T=1)

base_LDFs = {0: 2.0, 3: 1.5, 6: 1.2, 9: 1.1,
             12: 1.05, 15: 1.02, 18: 1.0}

claim_sim.simulate_claim_development(
    base_LDFs=base_LDFs,
    volatility=0.1,
    cumulative_factor=1.0,
)
```

```
print(claim_sim.claim_development.head())
```

Output interpretation. The `claim_development` DataFrame contains one row per (claim, development month) pair, with incurred losses that grow towards the ultimate value as the development age increases.

11.5 Example 5: Converting Simulated Claims to a Triangle

Purpose. Aggregate the simulated claim development data into a long-format loss triangle.

Code.

```
triangle_df = (
    claim_sim.claim_development
    .groupby(["accident_year", "development_month"])["incurred_loss"]
    .sum()
    .reset_index()
)
print(triangle_df.head())
```

Output interpretation. `triangle_df` is the long-format triangle ready for use with `chainladder`.

11.6 Example 6: Analysing the Triangle with `chainladder`

Purpose. Fit a chain-ladder model to the simulated triangle and obtain projected ultimate losses.

Code.

```
import chainladder as cl

triangle = cl.Triangle(
    triangle_df,
    origin="accident_year",
    development="development_month",
    columns="incurred_loss",
    cumulative=True,
)

model = cl.Chainladder().fit(triangle)
print(model.ultimate_)
```

Output interpretation. The projected ultimates can be compared against the sum of simulated ultimate losses by accident year. For a well-specified simulation and a sufficiently large sample, the two should agree within sampling error.

11.7 Example 7: Calculating VaR and TVaR

Purpose. Compute VaR and TVaR at multiple confidence levels from a simulation run.

Code.

```
from actsim import StochasticSimulator

simulator = StochasticSimulator(
    "poisson", (100,),
    "gamma", (2, 5000),
    num_sim=50000,
    keep_all=True,
    seed=99,
)

simulator.gen_agg_simulations()
report = simulator.analyze_results(quantiles=[0.5, 0.9, 0.95, 0.99, 0.995])
print(report.round(2))
```

Output interpretation. The output table contains VaR, TVaR, OEP, and AEP at each requested quantile. TVaR values exceed the corresponding VaR at the same level, as expected.

11.8 Example 8: Validating Simulated Frequency

Purpose. Compare the simulated mean claim count against the theoretical expected value.

Code.

```
import numpy as np
from actsim import StochasticSimulator

lambda_true = 50.0

simulator = StochasticSimulator(
    "poisson", (lambda_true,),
    "lognormal", (10, 0.5),
    num_sim=100000,
    keep_all=True,
    seed=2025,
)

simulator.gen_agg_simulations()

events_per_year = simulator.all_simulations.groupby("year").size()
print("Theoretical mean:", lambda_true)
print("Simulated mean:", events_per_year.mean())
```

Output interpretation. The simulated mean claim count should converge to $\lambda = 50$ as the number of simulations increases. Small deviations are due to simulation noise and are bounded by the standard error $\sqrt{\lambda/M}$, where M is the number of simulations.

Actuarial notes. This kind of internal consistency check should be standard practice when validating any simulation-based model. The validation chapter (Chapter [12](#)) discusses similar checks for severity, aggregate losses, and claim development.

Chapter 12

Validation and Testing

12.1 Why Validation Matters

A key objective of ActSim is to provide transparent and reproducible actuarial simulations. Without independent validation, simulation results are difficult to defend in audit, regulatory review, or peer review. This chapter describes the validation strategies that ActSim users should employ to verify that simulations behave as expected.

The validation strategies described here cover frequency, severity, aggregate losses, claim development, triangle reconciliation, and reproducibility. Together, they form a baseline that should be sufficient for open-source review and CAS-style scrutiny.

12.2 Frequency Distribution Validation

For frequency validation, simulated claim counts should be compared against the theoretical expected count.

For example, if claim frequency follows

$$N \sim \text{Poisson}(\lambda),$$

then $E[N] = \lambda$ and $\text{Var}(N) = \lambda$. The simulated mean and variance of the claim counts should converge to λ as the number of simulations M increases. The Monte Carlo standard error on the simulated mean is approximately $\sqrt{\lambda/M}$.

If a validation example uses $\lambda = 50$ with $M = 100,000$ simulations, the simulated average claim count should be close to 50, with a standard error of roughly $\sqrt{50/100000} \approx 0.022$. A simulated mean within a few standard errors of 50 is consistent with correct implementation.

For negative binomial frequencies, the analogous checks compare the simulated mean and variance against the theoretical $E[N]$ and $\text{Var}(N)$, where the variance exceeds the mean.

12.3 Severity Distribution Validation

For severity validation, the simulated samples should match the moments and quantiles of the target distribution. For lognormal severity with parameters μ and σ ,

$$E[X] = \exp\left(\mu + \frac{\sigma^2}{2}\right), \quad \text{Var}(X) = (\exp(\sigma^2) - 1) \cdot \exp(2\mu + \sigma^2).$$

The empirical mean, variance, and selected percentiles of the simulated severities should be close to these theoretical values. Larger sample sizes are required for accurate matching of upper percentiles.

A typical severity validation script:

```
import numpy as np
from actstats import actuarial as act

mu, sigma = 10, 0.5
n = 1_000_000

samples = act.lognormal(mu, sigma).rvs(size=n)

theoretical_mean = np.exp(mu + sigma**2 / 2)
print("Theoretical mean:", theoretical_mean)
print("Simulated mean:", samples.mean())
print("Simulated 95%:", np.percentile(samples, 95))
```

12.4 Aggregate Loss Validation

The compound expectation

$$E[S] = E[N] \cdot E[X]$$

provides a simple consistency check on aggregate loss simulations. The variance under independence is

$$\text{Var}(S) = E[N] \cdot \text{Var}(X) + \text{Var}(N) \cdot (E[X])^2.$$

Both quantities should be matched (within sampling error) by simulation output for a sufficient number of simulations.

When dependency is introduced via correlation or copulas, the simulated mean should remain close to $E[N] \cdot E[X]$ if the marginal distributions of N and X are unchanged. The variance, however, should change in a direction consistent with the imposed dependency.

12.5 Claim Development Validation

For each simulated claim with ultimate loss U and a deterministic LDF schedule $\{\text{LDF}_a\}$, the expected reported loss at age a is

$$E[L_a] = \frac{U}{\text{CDF}_a},$$

where CDF_a is the cumulative product of LDFs from age a to ultimate.

When stochastic LDFs are used, the same expectation holds approximately, because the multiplicative noise has mean 1. The simulated reported losses at each age should average to the deterministic pattern, with the spread controlled by the volatility parameter.

A simple validation check:

```
import pandas as pd

# After running simulate_claim_development:
expected_pattern = (
```

```

    claim_sim.claim_development
    .groupby("development_month")["incurred_loss"]
    .mean()
)
print(expected_pattern)

```

The mean reported loss should grow monotonically with development age and should approach the simulated ultimate at the latest age.

12.6 Triangle Reconciliation

Once the simulated claim development data have been aggregated into a loss triangle and analysed using `chainladder`, the projected ultimates can be reconciled against the underlying simulated ultimates.

```

ultimate_simulated = (
    claim_sim.claim_data
    .groupby(claim_sim.claim_data["incurred_date"].dt.year)["ultimate_loss"]
    .sum()
)

ultimate_chainladder = model.ultimate_

print("Simulated ultimate:", ultimate_simulated)
print("Chain-ladder ultimate:", ultimate_chainladder)

```

For a well-specified simulation with a sufficient number of claims, the two should agree within sampling error. Persistent differences indicate either a misspecification of the LDF schedule or a sample size that is too small for the chain-ladder method to converge.

12.7 Reproducibility Using Random Seeds

ActSim runs are fully deterministic given a random seed. The same input (policy table, distributional specification, seed) produces the same output across runs. This guarantee is achieved by passing the seed through to `numpy`'s random number generator at construction.

To verify reproducibility:

```

from actsim import StochasticSimulator

def run(seed):
    sim = StochasticSimulator(
        "poisson", (10,), "lognormal", (10, 0.5),
        num_sim=10000, keep_all=False, seed=seed,
    )
    sim.gen_agg_simulations()
    return sim.results.sum()

assert run(1234) == run(1234)

```

When sharing simulation results, the seed used to produce them should be recorded alongside the output.

12.8 Statistical Tests

In addition to mean-and-variance checks, the user may apply formal statistical tests:

- Kolmogorov–Smirnov test on simulated severities against the target CDF.
- Chi-square goodness-of-fit on simulated frequencies binned into integer counts.
- Anderson–Darling test for tail behaviour.

These tests are most informative when validating a particular distributional implementation. They are less useful for assessing the overall realism of a model relative to historical data, where domain judgement is essential.

12.9 Known Limitations

The current implementation has several limitations that users should be aware of:

- The Kolmogorov–Smirnov statistic is computed via `scipy.stats.kstest` on the data and the fitted CDF. For large sample sizes, this can take longer to compute than AIC or BIC.
- Chi-square statistics in `DistributionFitter` are computed by binning observed and expected frequencies; the choice of bin count affects the resulting statistic.
- The current copula implementation supports a fixed set of families (Gaussian, Frank, Gumbel, Clayton). Other families can be added but require a code change.
- Multivariate dependency simulation currently supports marginal distributions but does not retain claim-level data beyond the marginal totals. This limits some downstream validations.

These limitations are documented to allow reviewers to focus their scrutiny on the parts of the package that may behave unexpectedly. The maintainer welcomes contributions that address them.

Chapter 13

API Reference

This chapter provides a structured reference for the main classes and functions exposed by ActSim. It is intended for use as a lookup reference rather than a tutorial; readers who are new to ActSim should start with the chapters on the individual classes.

13.1 DistributionFitter

Constructor

```
DistributionFitter(data, distributions=None, metrics=None)
```

Argument	Description
<code>data</code>	Array-like of observed values to fit.
<code>distributions</code>	Optional list of distribution names. Defaults to all supported distributions.
<code>metrics</code>	Optional list of goodness-of-fit metrics. Defaults to ['aic', 'bic'].

Methods

Method	Description
<code>fit()</code>	Fits all candidate distributions to the data and computes goodness-of-fit metrics.
<code>select_best_fit()</code>	Internal method called by <code>fit()</code> ; selects the best fit under each requested metric.
<code>get_best_fit(metric)</code>	Returns the best-fit dictionary for the given metric.
<code>select_distribution(name)</code>	Selects a previously fitted distribution by name as the active distribution. Requires <code>fit()</code> to have been called.
<code>get_selected_dist()</code>	Returns the currently selected distribution object.
<code>get_selected_params()</code>	Returns the parameters of the currently selected distribution.
<code>predict(x)</code>	Evaluates the PDF (or PMF) of the selected distribution at <code>x</code> .

Method	Description
<code>sample(size=1)</code>	Draws random samples from the selected distribution.
<code>sample_mixed(zero_prop=0, one_prop=0, size=1)</code>	Draws samples with a given proportion of zeros and ones inserted.
<code>calculate_statistics()</code>	Returns mean, standard deviation, and key percentiles for both the data and the predicted density.
<code>plot_predictions(...)</code>	Plots a histogram of the data with the PDFs of the requested distributions overlaid. Argument: <code>distribution_names=None</code> .
<code>summary()</code>	Returns a DataFrame summarising the fitted distributions.
<code>truncate_data(self, remove_values=None, lower=None, upper=None, q_low=None, q_high=None, dropna=True, inplace=True)</code>	Truncate / clean data before distribution fitting.

13.2 StochasticSimulator

Constructor

```
StochasticSimulator(
    freq_dist, freq_params,
    sev_dist, sev_params,
    num_sim=10000,
    keep_all=False,
    seed=1,
    correlation=None,
    copula_type=None,
    theta=0,
)
```

Argument	Description
<code>freq_dist</code>	Name of the frequency distribution.
<code>freq_params</code>	Tuple of frequency distribution parameters.
<code>sev_dist</code>	Name of the severity distribution.
<code>sev_params</code>	Tuple of severity distribution parameters.
<code>num_sim</code>	Number of simulations or number of years in catastrophe modelling.
<code>keep_all</code>	If <code>True</code> , retain individual event records.
<code>seed</code>	Random seed.
<code>correlation</code>	Optional correlation between frequency and severity.
<code>copula_type</code>	Optional copula family.
<code>theta</code>	Copula dependence parameter.

Methods

Method	Description
<code>gen_agg_simulations()</code>	Runs the aggregate loss simulation.
<code>gen_multivariate_corr_simulations()</code>	Generates correlated marginal simulations across multiple lines of business. Arguments: <code>corr_matrix_file</code> , <code>dist_list_file</code> , <code>gen_marginal=False</code> .
<code>calc_agg_percentile(pct=95)</code>	Returns the requested percentile of aggregate losses.
<code>plot_distribution(bins=None, log_option=False)</code>	Plots a histogram of simulated aggregate losses.
<code>plot_correlated_variables()</code>	Plots simulated frequency–severity correlation.
<code>analyze_results(quantiles=[0.1, 0.25, 0.5, 0.75, 0.9, 0.95, 0.99], **kwargs)</code>	Returns VaR, TVaR, OEP, and AEP at the specified quantiles.
<code>apply_deductible_and_limit(...)</code>	Applies per-occurrence and aggregate deductibles and limits. Arguments: <code>per_occurrence_ded</code> , <code>per_occurrence_limit</code> , <code>agg_ded</code> , <code>agg_limit</code> .

Properties

Property	Description
<code>results</code>	Pandas Series of aggregate losses by simulation period.
<code>all_simulations</code>	Pandas DataFrame of individual event records (when <code>keep_all=True</code>).

13.3 ClaimSimulator

Constructor

```
ClaimSimulator(
    policies_df,
    random_seed=42,
    correlation=None,
    copula_type=None,
    copula_param=0,
)
```

Argument	Description
<code>policies_df</code>	Policy table; must contain the columns <code>policy_id</code> , <code>freq_dist</code> , <code>freq_params</code> , <code>sev_dist</code> , <code>sev_params</code> , <code>start_date</code> , and <code>end_date</code> . The <code>freq_params</code> and <code>sev_params</code> entries must be tuples.
<code>random_seed</code>	Random seed.
<code>correlation</code>	Optional frequency–severity correlation.
<code>copula_type</code>	Optional copula family.
<code>copula_param</code>	Copula dependence parameter.

Methods

Method	Description
<code>group_policies()</code>	Groups policies by frequency/severity specifications.
<code>simulate_claims()</code>	Simulates claim counts and severities for each policy group.
<code>simulate_dates_nhpp(lambda0=10, alpha=0.5, phase=0, T=1)</code>	Assigns occurrence dates using a non-homogeneous Poisson process.
<code>simulate_claim_development(...)</code>	Develops claims using stochastic LDFs. Arguments: <code>base_LDFs</code> , <code>volatility=0.1</code> , <code>cumulative_factor=1.0</code> .
<code>save_claim_development(filepath)</code>	Saves the development DataFrame to CSV.

Attributes

Attribute	Description
<code>policies</code>	Processed policy DataFrame.
<code>claim_data</code>	Simulated claim records.
<code>claim_development</code>	Long-format claim development data.

13.4 Config and load_config

load_config

```
load_config(file_path=None)
```

Convenience function that returns a `Config` object. If `file_path` is `None`, the default `config.yaml` shipped with the package is used.

Config

Method	Description
<code>has_key(key)</code>	Returns <code>True</code> if the key exists in the configuration.
<code>update(updates)</code>	Updates configuration values from a dictionary.
<code>clear()</code>	Removes all configuration values.
<code>reload()</code>	Re-reads the configuration file from disk.
<code>save()</code>	Writes the current configuration to file.

13.5 Utility Functions

timing_decorator

A decorator used internally to measure the runtime of long-running methods. It prints the elapsed time when the decorated function returns. This decorator is intended for development use rather

than as part of the public API.

13.6 Supported Distribution Names

The following table lists the distribution names accepted by both `DistributionFitter` and `StochasticSimulator`. The underlying implementations are provided by the `actstats` package (`actstats.actuarial`).

Name	Description
<code>normal</code>	Normal (Gaussian) distribution.
<code>logistic</code>	Logistic distribution.
<code>exponential</code>	Exponential distribution.
<code>gamma</code>	Gamma distribution.
<code>beta</code>	Beta distribution.
<code>lognormal</code>	Lognormal distribution.
<code>weibull</code>	Weibull distribution.
<code>pareto</code>	Pareto distribution.
<code>uniform</code>	Uniform distribution.
<code>poisson</code>	Poisson distribution.
<code>negative binomial</code>	Negative binomial distribution.

Parameter conventions follow those of `actstats`; see the `actstats` documentation for the exact parametrisation of each distribution.

Chapter 14

Contributing

14.1 Scope of Contributions

ActSim is intended to support open actuarial research and transparent simulation workflows. Contributions are welcome and may include:

- Bug fixes.
- Support for additional probability distributions.
- New goodness-of-fit metrics.
- Additional validation tests.
- Documentation improvements (including this manual).
- Worked examples and case studies.
- Integration examples with other actuarial packages.

The package is distributed under the Apache License 2.0, and all contributions are accepted under the same terms.

14.2 Setting Up a Development Environment

The recommended workflow uses an isolated virtual environment:

```
git clone https://github.com/jzhng105/actsim.git
cd actsim

python -m venv venv
source venv/bin/activate      # macOS / Linux
venv\Scripts\activate        # Windows

pip install -e .[dev]
```

Editable installation (`pip install -e`) ensures that any local changes to the source files are picked up immediately, without reinstallation.

14.3 Running the Test Suite

Tests are run using `pytest`:

```
pytest
```

When adding new functionality, please add corresponding tests under the `tests/` directory. Tests should be deterministic and run quickly.

14.4 Code Style

The code base follows standard Python conventions:

- PEP 8 formatting.
- Type hints on public method signatures where reasonable.
- Concise, descriptive names.
- Public methods documented with docstrings.

The repository's `black` configuration is the canonical source for formatting choices. Running `black .` before committing is recommended.

14.5 Adding a New Distribution

To add support for a new distribution:

1. Add the distribution to `actstats` (or confirm that an implementation already exists).
2. Register the distribution in the `available_distributions` dictionary in both `actfitter.py` and `actsimulator.py`.
3. Add the distribution to the `config.yaml` file under the appropriate category (severity or frequency).
4. Add a unit test that fits the new distribution to a known sample and verifies that the recovered parameters are close to the truth.

14.6 Pull Request Workflow

1. Fork the repository on GitHub.
2. Create a feature branch: `git checkout -b feature/short-description`.
3. Make your changes and add tests.
4. Run the test suite locally and ensure that it passes.
5. Commit your changes with a descriptive message.

6. Push the branch to your fork and open a pull request against the upstream repository's `main` branch.

The maintainer will review the pull request and request changes if necessary. Small, focused pull requests are easier to review than large omnibus changes.

14.7 Reporting Issues

Bug reports and feature requests should be filed via the GitHub issue tracker. Useful bug reports include:

- A short description of the unexpected behaviour.
- The version of ActSim, Python, and key dependencies.
- A minimal reproducible example.
- The actual and expected output.

14.8 Discussion and Roadmap

Open-ended discussion (design questions, methodology, roadmap) should take place on the GitHub Discussions tab rather than the issue tracker. The maintainer welcomes input on roadmap priorities, particularly from practising actuaries who can help align development with real-world needs.

Chapter 15

Citation

15.1 Citing ActSim

If you use ActSim in research, internal actuarial analysis, or educational materials, please cite the package. A general-purpose citation is:

Zhang, J. *ActSim: A Python-Based Actuarial Simulation Package*. GitHub repository, <https://github.com/casact/actsim>.

A BibTeX-style entry suitable for inclusion in academic work is:

```
@software{ActSim2025,  
  title = {ActSim: A Python package for actuarial risk modeling  
          and simulation},  
  author = {Juntao Zhang},  
  year = {2025},  
  url = {https://github.com/casact/actsim},  
}
```

If ActSim is later submitted to a CAS publication, e-Forum paper, or official research resource, this section will be updated to include the formal citation.

15.2 Citing Dependencies

ActSim relies on a number of open-source scientific Python libraries. Users who report results based on ActSim are encouraged to cite the underlying packages where appropriate, including:

- ActStats
- NumPy
- SciPy
- pandas
- matplotlib
- seaborn

- statsmodels
- chainladder (where used for reserving analyses)

Citing these dependencies acknowledges the work of the broader open-source ecosystem on which ActSim depends.

Chapter 16

Glossary

Accident year

The calendar year in which a claim occurred.

Aggregate loss

The total loss across all claims in a simulation period.

Aggregate Exceedance Probability (AEP)

The probability that aggregate (annual) losses exceed a given threshold. Reported as a function of return period in catastrophe risk applications.

Bornhuetter–Ferguson method

A reserving method that combines an a priori expected ultimate loss ratio with incurred development. Available in the `chainladder` package.

Chain-ladder method

A standard reserving technique that projects ultimate losses from a triangle by applying age-to-age development factors.

Claim development

The evolution of a claim’s incurred or paid amount between its date of occurrence and ultimate settlement.

Copula

A multivariate distribution function with uniform marginals, used to introduce dependency between random variables while preserving their marginal distributions.

Cumulative development factor (CDF)

The ratio of ultimate losses to losses at a given development age. The product of LDFs from a given age through to ultimate.

Development age

The number of months elapsed since the start of an accident period.

Frequency

The number of claims occurring during a specified period.

Goodness-of-fit metric

A statistic used to compare candidate distributions, such as AIC, BIC, log-likelihood, chi-square, or Kolmogorov–Smirnov.

Loss development factor (LDF)

The ratio of losses at one development age to losses at the previous age.

Monte Carlo simulation

A simulation technique that estimates the distribution of an outcome by repeated random sampling.

Non-homogeneous Poisson process (NHPP)

A Poisson process with a time-varying intensity function. Used in ActSim to assign occurrence dates to simulated claims with seasonal patterns.

Occurrence Exceedance Probability (OEP)

The probability that the largest single-event loss in a year exceeds a given threshold.

Paid loss

The cumulative amount paid on a claim at a given valuation date.

Random seed

An integer that initialises a pseudo-random number generator. Setting the seed makes simulations reproducible.

Incurred loss

The sum of paid losses and case reserves on a claim at a given valuation date.

Severity

The amount of loss associated with an individual claim.

Tail Value at Risk (TVaR)

The expected loss conditional on exceeding the VaR threshold, also known as the conditional tail expectation.

Triangle

A standard actuarial data layout in which rows correspond to accident periods and columns correspond to development ages.

Ultimate loss

The expected total cost of a claim once all development is complete.

Value at Risk (VaR)

The quantile of a loss distribution at a given confidence level.

Chapter 17

Appendix

17.1 Supported Distributions

The following table summarises the distributions supported by ActSim, their parameters as expected by the package, and typical actuarial usage.

Name	Parameters	Typical Usage
normal	(μ, σ)	General-purpose continuous fits, sensitivity analyses.
logistic	(μ, s)	Heavy-shouldered alternative to normal. follows wikipedia parameterization.
exponential	(β)	Inter-arrival times and severity benchmarks; follows Bahnemann's actuarial parameterization [bahnemann2015distributions].
gamma	(α, θ)	Common severity model with closed-form moments.
beta	(α, β)	Loss ratios, bounded severities. follows wikipedia parameterization.
lognormal	(μ, σ)	Most common severity model in property/-casualty.
weibull	(δ, β)	Survival-style models, alternative severity model; follows Bahnemann's actuarial parameterization [bahnemann2015distributions].
pareto	(α, β)	Heavy-tailed severities, large-loss modelling; follows Bahnemann's actuarial parameterization [bahnemann2015distributions].
uniform	(a, b)	Sensitivity testing, simple stylised examples.
poisson	(λ)	Standard frequency model.
negative binomial	(r, p)	Over-dispersed frequency model; follows wikipedia parameterization.

17.2 Parameter Conventions

The parameter conventions used by ActSim follow those of `actstats`, which provides actuarial-style parametrisations. For users who are familiar with `scipy.stats`, the differences are summarised below.

Distribution	Actuarial Parameters	SciPy Equivalent	Mean
lognormal	(μ, σ)	<code>lognorm($s = \sigma$, $scale = \exp(\mu)$)</code>	$\exp\left(\mu + \frac{\sigma^2}{2}\right)$
gamma	(α, β)	<code>gamma($a = \alpha$, $scale = \beta$)</code>	$\alpha\beta$
weibull	(δ, β)	<code>weibull_min($c = \delta$, $scale = \beta$)</code>	$\beta\Gamma\left(1 + \frac{1}{\delta}\right)$
pareto	(α, β)	<code>lomax($b = \alpha$, $scale = \beta$)</code>	$\frac{\beta}{\alpha - 1}, \alpha > 1$
beta	(α, β)	<code>beta($a = \alpha$, $b = \beta$)</code>	$\frac{\alpha}{\alpha + \beta}$
poisson	(λ)	<code>poisson($\mu = \lambda$)</code>	λ
negative_binomial	(r, p)	<code>nbinom($n = r$, $p = p$)</code>	$\frac{r(1-p)}{p}$
normal	(μ, σ)	<code>norm($loc = \mu$, $scale = \sigma$)</code>	μ
logistic	(μ, s)	<code>logistic($loc = \mu$, $scale = s$)</code>	μ
exponential	(β)	<code>expon($scale = \beta$)</code>	β
uniform	(a, b)	<code>uniform($loc = a$, $scale = b - a$)</code>	$\frac{a+b}{2}$
nonhomogeneous_poisson	$(\lambda_0, \alpha, \phi, T)$	custom NHPPDistribution	$\lambda_0 T + \frac{\lambda_0 \alpha}{2\pi} [\cos(\phi) - \cos(2\pi T + \phi)]$

Table 17.2: Actuarial parameterizations and SciPy equivalents for supported distributions.

When in doubt, validate the parameters by drawing a large sample, computing the empirical mean and variance, and comparing to the theoretical values for the chosen parametrisation.

17.3 Example Input Schemas

Policy Table for ClaimSimulator

Column	Type	Description
<code>policy_id</code>	integer / string	Unique policy identifier.
<code>freq_dist</code>	string	Frequency distribution name.
<code>freq_params</code>	tuple	Frequency distribution parameters.
<code>sev_dist</code>	string	Severity distribution name.
<code>sev_params</code>	tuple	Severity distribution parameters.
<code>start_date</code>	date / Timestamp	Policy effective date.

Column	Type	Description
end_date	date Timestamp	/ Policy expiration date.

Correlation Matrix CSV

For multivariate correlated simulation, the correlation matrix CSV should contain a header row of line-of-business names and a leading column of names. The example below corresponds to five lines of business.

```
Correlation Matrix, LoB1, LoB2, LoB3, LoB4, LoB5
LoB1, 1.0, 0.5, 0.5, 0.3, 0.2
LoB2, 0.5, 1.0, 0.5, 0.7, 0.3
LoB3, 0.5, 0.5, 1.0, 0.2, 0.5
LoB4, 0.3, 0.7, 0.2, 1.0, 0.3
LoB5, 0.2, 0.3, 0.5, 0.3, 1.0
```

Distribution List JSON

The companion JSON file specifies the marginal distributions:

```
[
  {"index": 1, "dist_name": "LoB1",
    "dist_type": "gamma",      "dist_param": [2, 1]},
  {"index": 2, "dist_name": "LoB2",
    "dist_type": "lognormal", "dist_param": [2, 1]},
  {"index": 3, "dist_name": "LoB3",
    "dist_type": "gamma",      "dist_param": [3, 1]},
  {"index": 4, "dist_name": "LoB4",
    "dist_type": "lognormal", "dist_param": [3, 2]},
  {"index": 5, "dist_name": "LoB5",
    "dist_type": "gamma",      "dist_param": [4, 2]}
]
```

The `dist_type` field uses the same names as the `actstats` distributions used elsewhere in the package.

17.4 Default Configuration File

The default `config.yaml` file shipped with ActSim is shown below for reference.

```
distributions:
  severity:
    - normal
    - logistic
    - exponential
    - gamma
    - beta
    - lognormal
    - weibull
```

```
- pareto
- uniform
frequency:
- poisson
- negative binomial

metrics:
- aic
- bic
- log_likelihood
- chisquare
```

17.5 Simulation Assumptions

The following assumptions apply to ActSim simulations unless the user makes alternative arrangements:

- Within a single simulation period, frequency and severity are independent unless a correlation or copula is specified.
- All severities within a period are independent and identically distributed.
- Across simulation periods, draws are independent.
- Stochastic LDFs are applied independently to each claim.
- Random seeds fully determine the simulation output, given the input data.

These assumptions are standard but may not match every modelling context. Users should review them when reporting results.

17.6 Known Limitations

A consolidated list of known limitations:

- Copula support is limited to Gaussian, Frank, Gumbel, and Clayton families.
- Multivariate correlated simulation does not retain claim-level data beyond marginal totals.
- Frequency/severity dependency is implemented via a bivariate correlation matrix only; higher-order dependency structures are not currently supported.
- KS statistic is computed via `scipy.stats.kstest` and uses the fitted CDF; for very large samples this can be computationally expensive.

17.7 Release History

Version	Date	Notes
0.1.0	2025	Initial public release. Includes <code>DistributionFitter</code> , <code>StochasticSimulator</code> , and <code>ClaimSimulator</code> .

Future releases will be documented in the project `CHANGELOG.md` file in the source repository.